

MATERI 7 – TRANSACT SQL

PEMROGRAMAN BASIS DATA S1 TI



TRANSACT SQL

Transact SQL (T-SQL) adalah jenis Bahasa untuk basis data SQL yang dikeluarkan oleh perusahaan Microsoft dan Sybase. T-SQL adalah bahasa pemrograman yang dikembangkan dari SQL, seperti diketahui SQL adalah bahasa non procedural, artinya alur program tidak seperti bahasa pemrograman biasa melainkan melalui **request** dan **response**. Pada system berbasis Microsoft, T-SQL ini diimplementasi pada perangkat lunak Microsoft SQL Server.

Melalui instruksi SQL, pengguna melakukan query atau transaksi, yang kemudian akan menerima jawaban dari database server berupa hasil atau *ResultSet*. Query dan transaksi dilakukan di bagian Client sedangkan pemrosesan dilakukan pada Server. T-SQL mengembangkan kemampuan SQL, sehingga T-SQL dapat melengkapi SQL dengan instruksi logic (procedural logic) yaitu program Aplikasi. Hasil proses SQL Server (*ResultSet*) dapat diolah lebih lanjut dengan menggunakan logic pemrograman procedural seperti Fungsi, Prosedur, Loop, Case, If Then Else dan lainnya.

Kerangka Transact SQL : Transact-SQL dimulai dengan deklarasi variable dan disusul dengan Blok Program.

VARIABLE DECLARATION

BLOK PROGRAM (WITH OR WITHOUT BEGIN – END)

VARIABLE

Pendefinisian variable T-SQL :

- Variabel harus dideklarasikan sebelum digunakan.
- Nama variable selalu dimulai dengan karakter @.
- Variabel dapat diberikan nilai melalui instruksi **SELECT** ataupun **SET**.

Contoh :

```
-- DECLARE
DECLARE
    @nama VARCHAR(35),
    @nilai INT
-- INPUT DATA
SET @nama = 'Stevi Ema W'
SELECT @nilai = 100
--OUTPUT DATA
PRINT 'Nama = ' + @nama + 'Nilai = ' + @nilai
```

SELECT dapat digunakan juga untuk eksekusi aritmatika atau lainnya, seperti contoh berikut :

```
SELECT @nilai = @nilai + 100
SELECT @tgl_sekarang = NOW
```

Variabel dapat digunakan untuk **mengambil nilai** dari sebuah query, contohnya adalah sebagai berikut :

```
DECLARE @nama VARCHAR(35)
SELECT @nama = name
FROM Customers
WHERE year_of_birth BETWEEN 2007 AND 2009
```

Contoh 2 :

```
DECLARE @gaji_maksimal INT
SELECT @gaji_maksimal = MAX(salary)
FROM Employee
```

PRINT dalam T-SQL

PRINT adalah fungsi yang menampilkan teks dan variable pada layar (*console*).

Contoh :

```
DECLARE @gaji_maksimal INT
SELECT @gaji_maksimal = MAX(salary)
FROM Employee
PRINT @gaji_maksimal
```

VARIABEL GLOBAL

Variabel global adalah variable yang disiapkan oleh SQL Server untuk memberikan informasi kepada Client. Variabel global bersifat **Read-Only**. Nama variable global diawali dengan @@.

Contoh :

@@ERROR

Bilangan bulat, yang menyatakan nomor error. Jika variable tersebut tidak sama dengan 0, maka system memberikan indikasi bahwa terjadi error dan identitas error dinyatakan dalam angka.

@@CURSOR_ROWS

Nilai balik dari cursor yang terakhir kali diperoleh.

@@TRANCOUNT

Mengembalikan jumlah baris yang berhasil dieksekusi pada transaksi

@@ROWCOUNT

Mengembalikan jumlah baris pada statement terakhir

STATEMENT IF

IF digunakan dalam mengendalikan alur program berdasarkan kondisi. Terdapat 3 model struktur IF dalam SQL Server, yaitu :

Model 1 :

```
IF condition
    Instruksi
```

Model 2 :

```
IF condition
    Instruksi 1
ELSE
    Instruksi 2
```

Model 3 :

```
IF condition
BEGIN
    Instruksi 1
    Instruksi 2
    . . . . .
END
```

Contoh 1 :

```
DECLARE
    @nilai INT,
    @rata_rata INT
SET @nilai = 80

SELECT @rata_rata = AVG(nilai)
FROM raport
WHERE mata_pelajaran = 'Matematika'

IF @nilai > @rata_rata
    PRINT 'Nilai diatas rata-rata'
ELSE
    PRINT 'Nilai kurang dari rata-rata'
```

WHILE

While digunakan dalam mengeksekusi satu blok program berulang-ulang sampai kondisi pada WHILE menjadi **FALSE**.

Struktur WHILE :

```
WHILE condition
BEGIN
    . . . . (statement)
END
```

Contoh :

```
DECLARE @batas INT
SET @batas = 5

WHILE @batas > 0
BEGIN
    PRINT 'data ke - ' + str(@batas)
    SELECT @batas = @batas - 1
END
```

Fungsi **str()** digunakan untuk mengkonversi bilangan integer menjadi **string**.

CONTINUE DAN BREAK

Continue dan Break berkaitan dengan WHILE. Continue melanjutkan alur program pada pemeriksaan kondisi WHILE, sedangkan BREAK menghentikan alur program dari WHILE.

Struktur :

```
WHILE condition
BEGIN
    . . . . .
    IF condition2
        CONTINUE
    . . . . .
END
```

RETURN STATEMENT

Return akan menghentikan program dari eksekusi. RETURN dapat digunakan dalam memproses ERROR.

```
IF @@ERROR <> 0
BEGIN
    PRINT 'Error'
    RETURN
END
```

CASE STATEMENT

Case menyederhanakan IF yang berlapis dengan tujuan agar program dapat dimengerti/dibaca dengan lebih mudah.

SINTAK :

```
CASE
    WHEN condition1 THEN
    WHEN condition2 THEN
    WHEN condition3 THEN
    ELSE
END
```

Contoh :

```
DECLARE
    @huruf CHAR(1),
    @hasil VARCHAR(50)

SELECT @huruf = 'A'
SELECT @hasil =
CASE
    WHEN @huruf = 'A' THEN 'Sangat baik'
    WHEN @huruf = 'B' THEN 'Baik'
    WHEN @huruf = 'C' THEN 'Cukup'
    ELSE
        'Belajar lagi'
END
PRINT 'Hasilnya adalah ' + @hasil
```

CURSOR

Cursor adalah sejenis variable yang digunakan menampung banyak nilai berupa baris atau *record*.

Untuk menggunakan cursor ada beberapa tahapan, yaitu :

1. Deklarasi (DECLARE)
2. Buka cursor (OPEN)
3. Ambil data (FETCH)
4. Tutup cursor (CLOSE)
5. Membersihkan memory reference (DEALLOCATE)

Contoh :

```
--mendeklarasikan variabel
DECLARE @name VARCHAR(30)

--mendeklarasikan cursor
DECLARE customer_data CURSOR FOR
SELECT name FROM Customers

--membuka cursor
OPEN customer_data

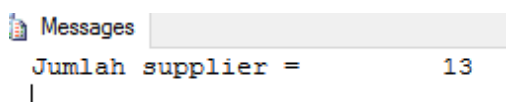
--menentukan posisi cursor
FETCH NEXT FROM customer_data INTO @name

--perulangan untuk menampilkan data
WHILE (@@FETCH_STATUS = 0)
BEGIN
    PRINT @name
    FETCH NEXT FROM customer_data INTO @name
END

--menutup cursor
CLOSE customer_data
DEALLOCATE customer_data
```

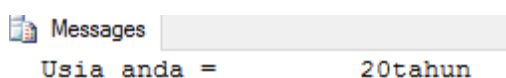
TASK !

1. Dengan menggunakan variable, tampilkan jumlah supplier yang beralamat Jakarta.



```
Messages
Jumlah supplier =      13
|
```

2. Dengan memanfaatkan variable, buat sebuah program untuk menghitung berapa usia anda saat ini. Hasil output seperti contoh berikut :



```
Messages
Usia anda =      20tahun
```

3. Sama seperti soal sebelumnya dengan menggunakan variabel, tampilkan data barang yang memiliki harga jual tertinggi (*selling_price*).

```
Messages
Kode barang = CG-006-IT000035
Nama barang = Tupperware Blossom Series
```

4. Buat transact SQL untuk menampilkan nilai factorial dari sebuah bilangan.

```
Messages
Nilai faktorial dari 5
adalah 120
```

5. Buat transact SQL untuk menentukan bilangan tersebut bilangan genap atau bilangan ganjil

```
Messages
bilangan 7 adalah bilangan ganjil
```

6. Jalankan query berikut ini :

```
IF DATEPART(dw, GETDATE()) = 7 OR DATEPART(dw, GETDATE()) = 1
    PRINT 'It is the weekend.'
ELSE
    PRINT 'It is a weekday.'
```

Jelaskan maksud dari query tersebut !

7. Dengan menggunakan CURSOR tampilkan data customer yang pernah melakukan pembelian barang.

```
Messages
CUS-000002 - Eka Kharisma
CUS-000005 - Nurafni Oktaviani
CUS-000001 - Bagus
CUS-000004 - Yudha Adisaputra
CUS-000010 - Ervina Jayanti
CUS-000007 - Aulia Luthfi Anna
CUS-000003 - Aliany Milawaty
CUS-000008 - Ichzam Rusmana
CUS-000006 - Meilia
CUS-000009 - Tri Yulianti
```

8. Dengan menggunakan CURSOR tampilkan data 3 data barang yang memiliki stok paling banyak

```
Messages
CG-001-IT000002 - Roberto Cavalli Sunglass
CG-002-IT000011 - Black Gray Long Sleeved Shirt
CG-002-IT000013 - Sweater Tribal Gazilo
```

LAMPIRAN FUNCTION

STRING FUNCTION

Nama Fungsi	Keterangan
ASCII	Mengembalikan nilai kode ASCII. ASCII (character_expression) Contoh: <code>SELECT ASCII('A')</code>
CHAR	Kebalikan dari ASCII, yaitu mengembalikan character dari ASCII. CHAR (integer_expression) Contoh: <code>SELECT CHAR(65)</code>
LEN	Mengembalikan panjang dari suatu string. LEN (string_expression) Contoh: <code>SELECT MaterialDescription, LEN(MaterialDescription) Panjang FROM MATERIAL</code>
LEFT	Mengambil part dari suatu string dengan panjang yang spesifik. LEFT (character_expression , integer_expression) Contoh: <code>SELECT LEFT(CustomerName,3) FROM CUSTOMER</code>
RIGHT	Mengambil part dari suatu string dengan panjang yang spesifik. RIGHT (character_expression , integer_expression) Contoh: <code>SELECT RIGHT(CustomerName,3) FROM CUSTOMER</code>
SUBSTRING	Mengambil part dari character SUBSTRING (expression ,start , length) Contoh: <code>SELECT SUBSTRING('LPKIA JAYA',1,5)</code>
LOWER	Mengembalikan string dengan huruf besar (Kapital) LOWER (character_expression) Contoh: <code>SELECT LOWER('INI HURUF BESAR')</code>
UPPER	Mengembalikan string dengan huruf kecil UPPER (character_expression) Contoh: <code>SELECT UPPER('ini huruf kecil')</code>
LTRIM	Mengembalikan character dengan menghapus spasi pada character sebelumnya (kiri) LTRIM (character_expression) Contoh: <code>SELECT LTRIM(' No one is secure')</code>
RTRIM	Mengembalikan character dengan menghapus spasi pada character setelah (kanan) RTRIM (character_expression) Contoh: <code>SELECT RTRIM('No one is secure ')</code>
REVERSE	Mengembalikan dengan character terbalik REVERSE (character_expression) Contoh: <code>SELECT REVERSE('kebalik')</code>
REPLICATE	Mengulang string dengan spesifik number REPLICATE (string_expression ,integer_expression) Contoh: <code>SELECT REPLICATE('6',3)</code>
SPACE	Mengulang spase dengan spesifik number SPACE (integer_expression) Contoh: <code>SELECT SPACE(5)+'lima spasi'</code>
STR	Konversi dari numeric ke string STR (float_expression [, length [, decimal]]) Contoh: <code>SELECT STR(10000)</code>
REPLACE	Merubah spesifik suatu string dengan string lain REPLACE (string_expression1, string_expression2 , string_expression3) Contoh: <code>SELECT REPLACE('UJUNG BERUNG','BERUNG','BRONX')</code>

SYSTEM FUNCTION

Nama Fungsi	Keterangan
ISDATE	Memeriksa sebuah ekspresi tanggal dan Mengembalikan nilai integer 1 jika valid 0 jika tidak valid ISDATE (expression) Contoh: <code>SELECT ISDATE('10/10/2012')</code>
ISNULL	Mengganti NULL dengan spesifik value ISNULL (check_expression , replacement_value) Contoh: <code>SELECT ISNULL(CustomerName, 'Unknown') FROM Customer</code>
ISNUMERIC	Memeriksa sebuah ekspresi numeric dan Mengembalikan nilai integer 1 jika valid 0 jika tidak valid ISNUMERIC (expression) Contoh: <code>SELECT ISNUMERIC('a1000')</code>
NULLIF	Mengembalikan value NULL jika kedua ekspresi sama NULLIF (expression1 , expression2) Contoh: <code>SELECT NULLIF(CustomerName, 'PT. ABC') FROM Customer</code>

DATE FUNCTION

Nama Fungsi	Keterangan																								
GETDATE	Mengembalikan tanggal sekarang dari server dimana SQL terinstall Contoh: <code>SELECT GETDATE()</code>																								
DAY	Mengembalikan nilai integer part tanggal dari tanggal DAY (date) Contoh: <code>SELECT DAY(GETDATE())</code>																								
MONTH	Mengembalikan nilai integer bulan dari tanggal MONTH (date) Contoh: <code>SELECT MONTH(GETDATE())</code>																								
YEAR	Mengembalikan nilai integer tahun dari tanggal YEAR (date) Contoh: <code>SELECT YEAR(GETDATE())</code>																								
DATEADD	Mengembalikan datetime baru berdasarkan penambahan interval ke specific tanggal DATEADD (datepart , number, date) Contoh: <code>SELECT DATEADD(day, 5, '02/28/2012')</code> Dimana datepart: <table border="1"> <thead> <tr> <th>Date Part</th><th>Singkatan</th></tr> </thead> <tbody> <tr> <td>year</td><td>yy, yyyy</td></tr> <tr> <td>quarter</td><td>qq, q</td></tr> <tr> <td>month</td><td>mm, m</td></tr> <tr> <td>dayofyear</td><td>dy, y</td></tr> <tr> <td>day</td><td>dd, d</td></tr> <tr> <td>week</td><td>wk, ww</td></tr> <tr> <td>weekday</td><td>dw, w</td></tr> <tr> <td>hour</td><td>Hh</td></tr> <tr> <td>minute</td><td>mi, n</td></tr> <tr> <td>second</td><td>ss, s</td></tr> <tr> <td>millisecond</td><td>Ms</td></tr> </tbody> </table>	Date Part	Singkatan	year	yy, yyyy	quarter	qq, q	month	mm, m	dayofyear	dy, y	day	dd, d	week	wk, ww	weekday	dw, w	hour	Hh	minute	mi, n	second	ss, s	millisecond	Ms
Date Part	Singkatan																								
year	yy, yyyy																								
quarter	qq, q																								
month	mm, m																								
dayofyear	dy, y																								
day	dd, d																								
week	wk, ww																								
weekday	dw, w																								
hour	Hh																								
minute	mi, n																								
second	ss, s																								
millisecond	Ms																								

DATEDIFF	Mengembalikan number dari batasan tanggal dan waktu diantara dua tanggal DATEDIFF (datepart , startdate , enddate) Contoh: <code>SELECT (DATEDIFF(Day, '04/21/1983', GETDATE()))/365 Taun</code>
DATEPART	Mengembalikan nilai integer bagian dari tanggal DATEPART (datepart , date) Contoh: <code>SELECT DATEPART(month, GETDATE()) AS 'Bulan'</code>
DATENAME	Mengembalikan string datepart dari sebuah specific tanggal DATENAME (datepart ,date) Contoh: <code>SELECT DATENAME(month, GETDATE()) AS 'Nama Bulan';</code>

NUMERIC FUNCTION

Nama Fungsi	Keterangan
ABS	Mengembalikan value positive dari ekspresi numeric Contoh: <code>ABS (numeric_expression)</code> <code>SELECT ABS(-1)</code>
CEILING	Mengembalikan pembulatan kebawah, lebih besar atau samadengan ekpresi numeric <code>CEILING (numeric_expression)</code> Contoh: <code>SELECT CEILING(123.45)</code>
FLOOR	Mengembalikan pembulatan keatas, lebih kecil atau samadengan ekpresi numeric <code>FLOOR (numeric_expression)</code> Contoh: <code>SELECT GETDATE()</code>
ROUND	Pembulatan dengan panjang presisi tertentu <code>ROUND (numeric_expression , length [,function])</code> Contoh: <code>SELECT ROUND(123.345, 0)</code>
SQRT	Mengembalikan nilai akar dari value float <code>SQRT (float_expression)</code> Contoh: <code>SELECT SQRT(16)</code>
POWER	Mengembalikan pangkat dari value flot <code>POWER (float_expression , y)</code> Contoh: <code>SELECT POWER(2,4)</code>

KONVERSI

Digunakan untuk mengkonversi dari satu tipe data ke tipe data lain

Konversi dengan CAST

```
CAST ( ekspresi AS data type [ (panjang) ] )
```

Contoh :

```
SELECT CAST(getdate() as varchar(10))
```

```
SELECT CAST('10' as int)+CAST('5' as int)
```

Konversi dengan CONVERT

```
CONVERT ( data type [ ( panjang ) ] , ekspresi [ , style ] )
```

Contoh :

```
SELECT CONVERT(varchar,getdate(),103)
```

```
SELECT CONVERT(int,'10' )+CONVERT(int,'5' )
```